

EE332

Digital System Design

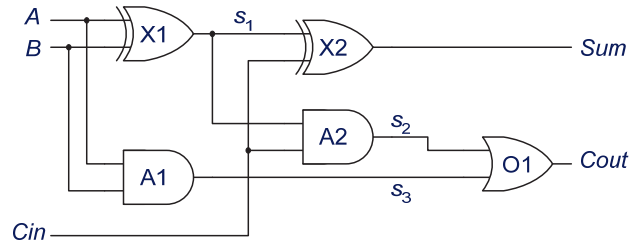
Dr. Yi Zeng
Assistant Professor,
Department of Electrical and Electronic Engineering
Office: S236, South Tower of COE
Email: zengyi@sustech.edu.cn

```

library IEEE;
use IEEE.std_logic_1164.all;

entity full_adder is
port (A, B, Cin : in std_logic;
      Sum, Cout: out std_logic);
end entity full_adder;

```



```

architecture dataflow of full_adder is
    signal s1, s2, s3 : std_logic;
    constant gate_delay : time := 10 ns;
begin
    L1: s1 <= (A xor B) after gate_delay;
    L2: s2 <= (Cin and s1) after gate_delay;
    L3: s3 <= (A and B) after gate_delay;
    L4: sum <= (s1 xor Cin) after gate_delay;
    L5: cout <= (s2 or s3) after gate_delay;
end architecture dataflow;

```

type time

} Architecture
declarative
segment

} Architecture
body

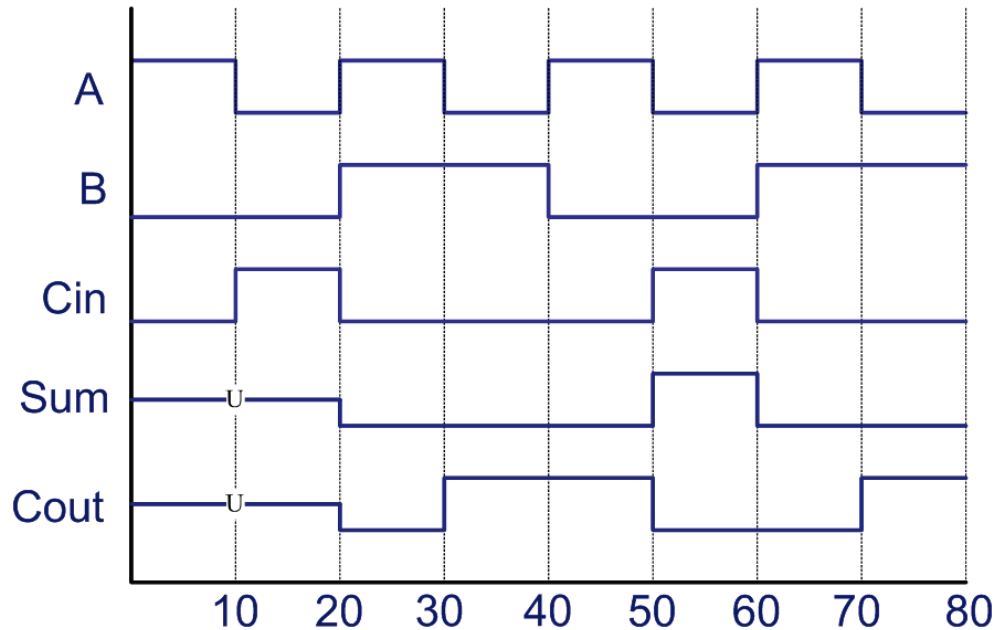
constant object



```

L1: s1 <= (A xor B) after gate_delay;
L2: s2 <= (Cin and s1) after gate_delay;
L3: s3 <= (A and B) after gate_delay;
L4: sum <= (s1 xor Cin) after gate_delay;
L5: cout <= (s2 or s3) after gate_delay;

```

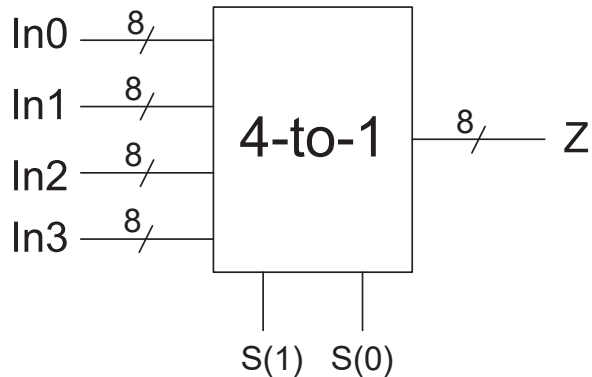


3.2 Conditional signal assignment

- Simple CSAs is convenient for describing gate-level circuits whose behavior can be expressed with Boolean equations.
- It is useful to model circuits at higher levels of abstraction such as multiplexors, decoders, priority encoders, etc.

4-to-1, 8-bit multiplexor

Input S	Output Z
00	In0
01	In1
10	In2
11	In3



```

library IEEE;
use IEEE.std_logic_1164.all;

entity mux4 is
port (In0, In1, In2, In3 : in std_logic_vector (7 downto 0);
      S: in std_logic_vector(1 downto 0);
      Z : out std_logic_vector (7 downto 0) );
end entity mux4;

```

```

architecture con_arch1 of mux4 is
begin
    Z <= In0 when S = "00" else
        In1 when S = "01" else
        In2 when S = "10" else
        In3;
end architecture con_arch1;

```

```

architecture con_arch2 of mux4 is
begin
    Z <= In0 when S = "00" else
        In1 when S = "01" else
        In2 when S = "10" else
        In3 when S = "11" else
        "XXXXXXXXX";
end architecture con_arch2;

```

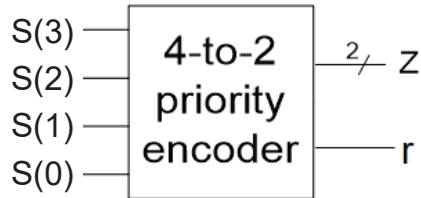
- The conditional signal assignment itself is a concurrent signal assignment.
- It has one target, but can have more than one expression.

General form:

```
target <=  {expression} when {condition} else
           {expression} when {condition} else
           .....
           {expression};
```

- When this CSA is executed, the expressions in the right hand side are evaluated in the order that they appear.
 - The order of the expressions with their respective conditions inside the statement is important.
- Even there are several lines of text, this corresponds to only one signal assignment statement.

4-to-2 priority encoder



Assume that S(3) has the highest priority

Input S	Output Z, r
1 ---	11, 1
0 1 --	10, 1
0 0 1 -	01, 1
0 0 0 1	00, 1
0 0 0 0	00, 0

```
library IEEE;
use IEEE.std_logic_1164.all;

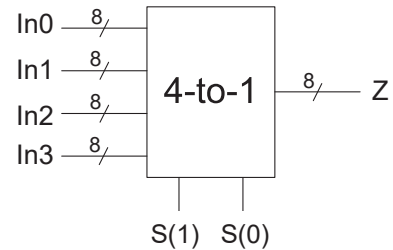
entity pr_encoder is
port (S: in std_logic_vector(3 downto 0);
      Z : out std_logic_vector (1 downto 0);
      r : out std_logic);
end entity pr_encoder;
```

```
architecture con_behavioral of pr_encoder is
begin
  Z <= "11" when S(3) = '1' else
        "10" when S(2) = '1' else
        "01" when S(1) = '1' else
        "00" when S(0) = '1' else
        "00";
  r <= S(3) or S(2) or S(1) or S(0);
end architecture con_behavioral;
```

3.3 Selected signal assignment

- It is similar to the conditional signal assignment statement.
- The value of a target signal is determined by the value of a select expression.
- Example: the same 4-1 multiplexer.

```
architecture sel_behavioral of mux4 is  
begin  
    with S select  
        Z <= In0 when "00",  
            In1 when "01",  
            In2 when "10",  
            In3 when others;  
end architecture sel_behavioral;
```

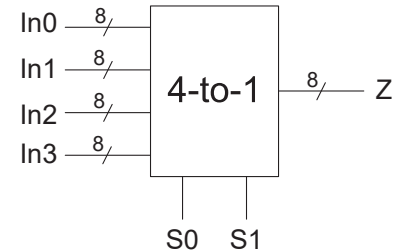


with S select

```
Z <= In0 when "00"  
      In1 when "01",  
      In2 when "10",  
      In3 when "11";
```

X

Why?



when要包含所有的情况
且when后的条件是互斥的
即有且仅有一个条件同时发生

with S select

```
Z <= In0 when "00",  
      In1 when "01",  
      In2 when "10",  
      In3 when "11",  
      "XXXXXXXX" when others;
```

✓

4-to-2 priority encoder based on a selected signal assignment statement

```
architecture sel_arch of pr_encoder is
begin
```

```
  with S select
```

```
    Z <= "11" when "1000" | "1001" | "1010" | "1011" |
          "1100" | "1101" | "1110" | "1111",
          "10" when "0100" | "0101" | "0110" | "0111",
          "01" when "0010" | "0011",
          "00" when others;
```



```
end architecture behavioral
```

S(3)

S(2)

S(1)

S(0)

4-to-2
priority
encoder

z Z

Why?

假如S为1000，程序运行时，编译器会认为1000不等于1---，从而进入others的情况

```
architecture sel_arch of pr_encoder is
begin
```

```
  with S select
```

```
    Z <= "11" when "1---",
          "10" when "01--",
          "01" when "001-",
          "00" when others;
```



```
end architecture behavioral
```

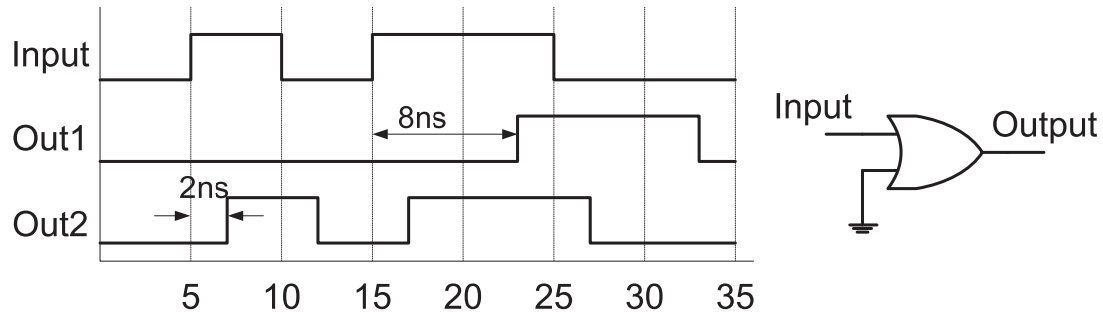
- The choices for the select expression are not evaluated in sequence.
- All choices are evaluated, but one and only one must be true.
- All of the choices that the programmer specifies must cover all the possible values of the select expression.
- When an event occurs on a signal used in the select expression, or any of the signals used in one of the choices, the statement is executed.

3.4 Understanding delays

- Accurate representation of the behavior of digital circuits requires accurate modeling of delays through the various components.
- Three types of delay models
 - Inertial delay model
 - Transport delay model
 - Delta delay

3.4.1 The inertial delay model

- Digital circuits takes a finite amount of time for the output of a gate to respond to a change on the input.
- This implies that the change on the input has to persist for a certain period of time to ensure that the output will response.
- If it does not persist long enough the input events will not be propagated to the output.
- This propagation delay model is referred to as the inertial delay model.



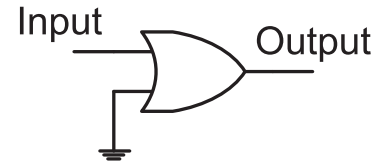
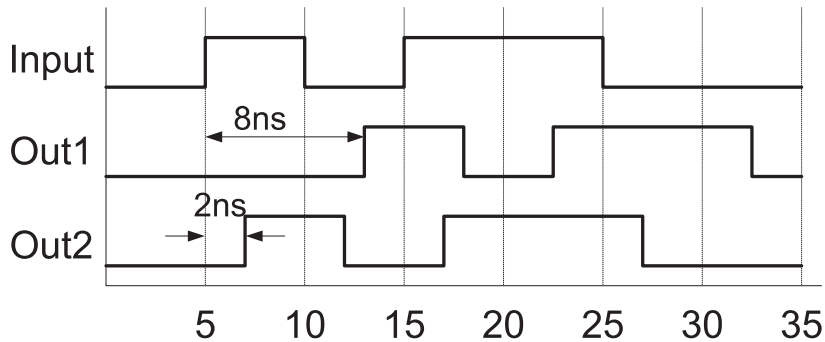
```
Out1 <= Input or '0' after 8 ns;
Out2 <= Input or '0' after 2 ns;
```

- Any pulse in the input with a width of less than the propagation delay through the gate is said to be rejected.
- often used for component delays
- Default in VHDL program

默认为惯性delay

3.4.2 The transport delay model

- Transport delay models the delays in hardware that do not exhibit any inertial delay.
- This delay represents pure propagation delay; that is, any changes on an input are transported to the output, no matter how small of the width, after the specified delay.
- Keyword **transport** is used in a signal assignment statement for transport delay model.

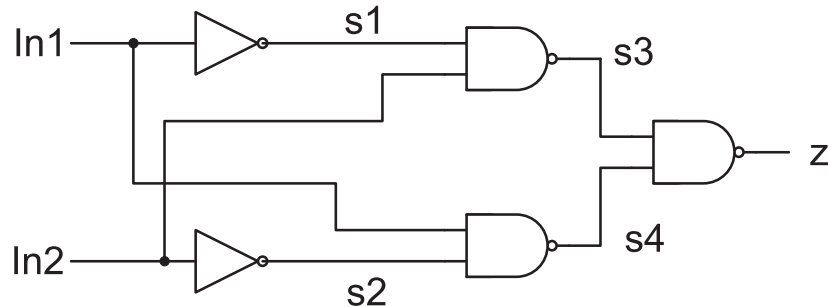


```
Out1 <= transport (Input or '0') after 8 ns;  
Out2 <= transport (Input or '0') after 2 ns;
```

- Pulses are propagated, irrespective of width
- good for interconnect delays.

3.4.3 Delta Delays

- If we do not specify a delay for the occurrence of an event on a signal, for example
 `sum <= (x xor y);`
 a **delta delay** is assumed by the simulator
- Delta delay is an infinitesimally small delay, Δ .
- In a signal assignment, the value is not assigned to the signal directly but after a delta delay at the earliest.
- Delta delays are simply used to enforce dependencies between events and thereby ensure correct simulation.



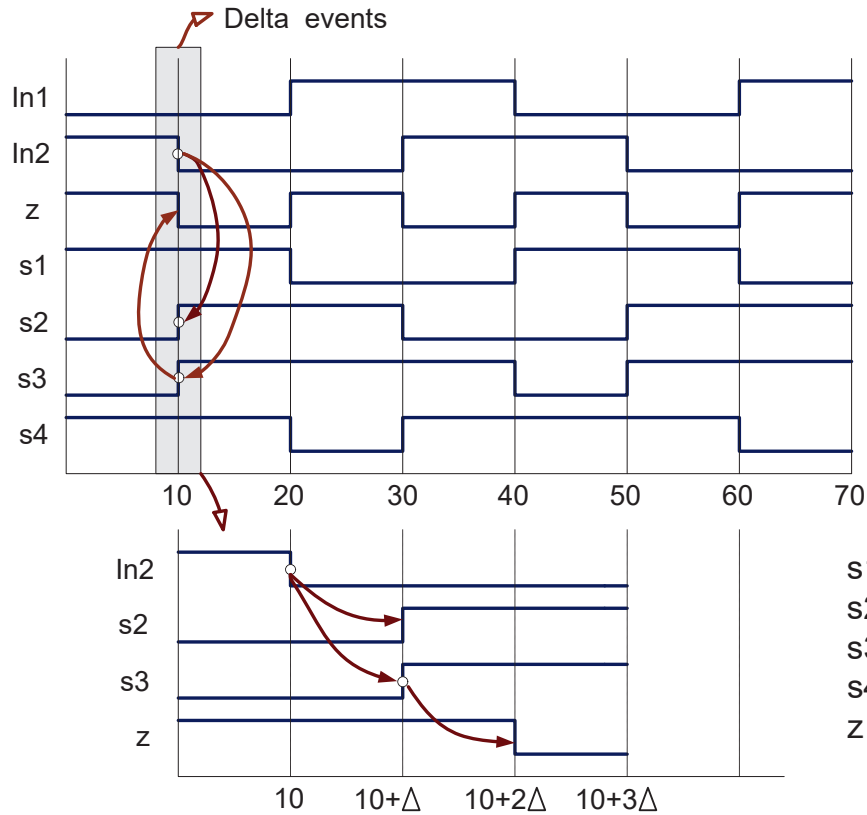
```

library IEEE;
use IEEE.std_logic_1164.all;

entity combinational is
  port (In1, In2 : in std_logic;
        z : out std_logic);
end entity combinational;
  
```

```

architecture behavior of combinational is
  signal s1, s2, s3, s4 : std_logic := 0;
begin
    s1 <= not In1;
    s2 <= not In2;
    s3 <= not (s1 and In2);
    s4 <= not (s2 and In1);
    z <= not (s3 and s4);
end architecture behavior;
  
```

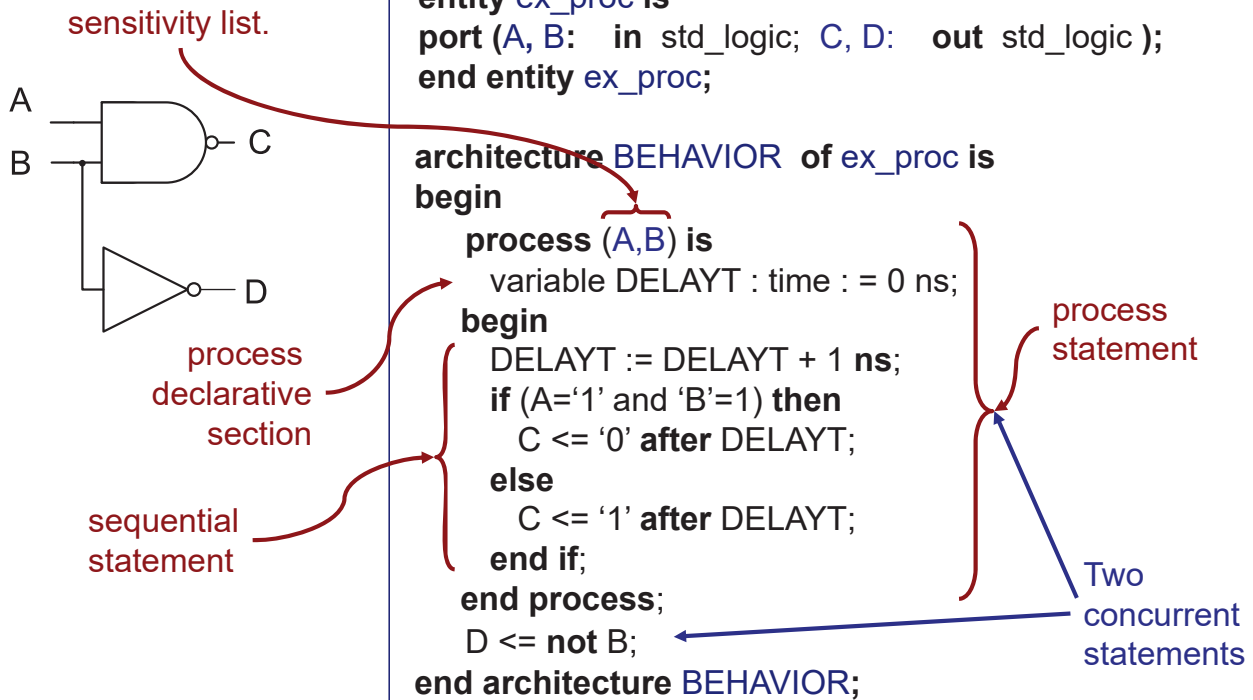


Chapter 4. Sequential VHDL

- Sequential statements are specified inside a process.
- Processes represent fundamental method by which concurrent activities are modeled.

不同的process之间是并行的，每一个process中是顺序执行的

4.1. The process construct



- A process is a large concurrent statement that itself contains a series of sequential statements.
- The body of the process is treated by VHDL as a single concurrent statement and is executed at the same time as all other concurrent statements in the simulation.

Process with a sensitivity list

- The sensitivity list is a list of signal to which the process responds.

```
signal a, b, c, y: std_logic; -- in architecture declaration
...
process (a, b, c) -- in architecture body
begin
    y <= a and b and c; -- infer a combinational circuit
end process;
```

```
signal a, b, c, y: std_logic;
...
process (a) -- a process with incomplete sensitivity list
            -- not infer a combinational circuit
begin
    y <= a and b and c;
end process;
```

⚡ sensitivity list中的信号发生变化时，对应process将会被执行

Process with a wait statement

- A process with wait statements has one or more wait statement but no sensitivity list.

wait on signals;

wait until boolean_expression;

wait for time_expression;

```
process
begin
  y <= a and b and c;
  wait on a, b, c;
end process;
```

In synthesis, only few well defined forms of wait statement can be used, and normally only one wait statement is allowed in a process.
Not recommended.



```
process (a, b) is
begin
  c <= a and b;
end process;
```

sensitivity list. The process will be activated if either a or b (or both) changes states

```
Process is
begin
  c <= a and b;
  wait on a, b;
end process;
```

A process can have wait statement or sensitivity list, but not both

```
process is
begin
  c <= a and b;
end process;
```

} This is an infinite loop.

4.2. Sequential statements

- sequential signal assignment statement
- variable assignment statement
- if statement
- case statement
- null statement
- wait statement
- loop statement
- exit statement
- next statement

4.2.1 Signal assignment statement

Signal-object <= *expression* [**after** *delay-value*];

- Outside a process – concurrent signal assignment statement
- Within a process – sequential signal assignment, executed in sequence with respect to the other sequential statement which appear within that process.
- When a signal assignment statement is executed, the value of expression is computed immediately, and this computed value is scheduled to be assigned to the signal after the specified delay.

- Inside a process, a signal can be assigned multiple times. If all assignments are with δ -delay, only the last assignment takes effect.

```
signal a, b, c, d, y: std_logic;  
...  
process(a, b, c, d)  
begin  
    y <= a or c;  
    y <= a and b;  
    y <= c and d;  
end process;
```



```
signal a, b, c, d, y: std_logic;  
...  
process(a, b, c, d)  
begin  
    y <= c and d;  
end process;
```

- Although this segment is easy to understand, multiple assignment may introduce subtle mistakes in a more complex code and make synthesis very difficult.

注意：只会延时一个delta，VHDL中执行和延时是分开的两个进程，执行无时延在进程内的信号是顺序处理，并行赋值的

- Unless there is a compelling reason, it is a good idea to avoid assign a signal multiple times in a process. The **only exception** is the assignment of a default value in the if and case statement.
- The result will be very different if the multiple assignments are the concurrent signal assignment statements

```
signal a, b, c, d, y: std_logic;  
...  
-- the statements are not inside a process  
y <= a or c;  
y <= a and b;  
y <= c and d;
```

- The above code is syntactically correct.
- However, the design is incorrect because of the potential output conflict.

4.2.2 Variable assignment statement

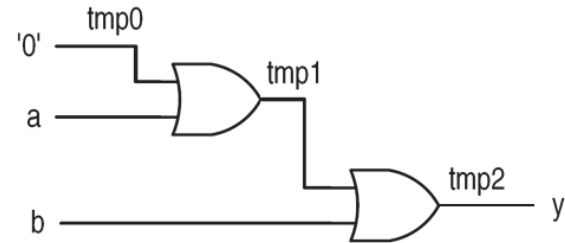
- Variables can be declared and used inside a process statement.
- Variable assignment statement:
variable-object := expression;
- Variable assignment is immediate.

```
Signal a, b, y : std_logic;  
.....  
process (a, b) is  
    variable temp : std_logic;  
    begin  
        temp := '0';  
        temp := temp or a;  
        temp := temp or b;  
        y <= temp;  
    end process;
```

Although the behavior of a variable is easy to understand, mapping it to hardware is difficult. **Not recommended.**

❑ To realize the previous process in hardware, we have to rename the variables.

```
process (a, b) is  
  variable tmp0, tmp1, tmp2 : std_logic;  
  begin  
    tmp0 := '0';  
    tmp1 := tmp0 or a;  
    tmp2 := tmp1 or b;  
    y <= tmp2;  
  end process;
```



For the lack of clear hardware mapping, we should try to use signal in code in general and resort to variable only for the characteristic that cannot be described by signals.

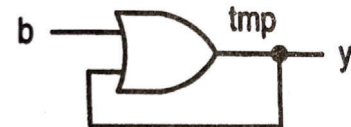
❑ For comparison purpose

```
Signal a,b,y,tmp : std_logic;  
process (a, b, tmp) is  
  begin  
    tmp <= '0';  
    tmp <= tmp or a;  
    tmp <= tmp or b;  
    y <= tmp;  
  end process;
```



```
Signal a,b,y,tmp : std_logic;  
process (a, b, tmp) is  
  begin  
    tmp <= tmp or b;  
    y <= tmp;  
  end process;
```

Not recommended.



4.2.3 If statement

- An if statement selects a sequence of statements for execution based on the value of a condition.
- General form of an if statement

 **if** *condition* **then**
 sequential-statements
{ elseif *condition* **then**
 sequential-statements **}**
[else
 sequential-statements **]**
end if;

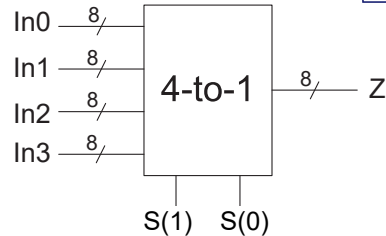
```
if SUM <=100 then  
    SUM := SUM +100;  
end if;
```

```
if NICKLE_IN then  
    DEPOSITED <= T5;  
elseif DIME_IN then  
    DEPOSITED <= T10;  
elseif QUARTER_IN then  
    DEPOSITED <= T25;  
else DEPOSITED <=Terror;  
end if;
```

Note the absence of the letter 'e' in elseif !

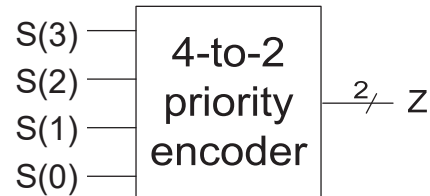
4-to-1 multiplexer based on an if statement

```
architecture if_arch of mux4 is
begin
  process(In0, In1, In2, In3, S)
  begin
    if (S = "00") then
      Z <= In0;
    elsif (S = "01") then
      Z <= In1;
    elsif (S = "10") then
      Z <= In2;
    else Z <= In3;
    end if;
  end process;
end if_arch;
```



4-to-2 priority encoder based on an if statement

```
architecture if_arch of pr_encoder is
begin
  process(S)
  begin
    if (S(3) = '1') then
      Z <= "11";
    elsif (S(2) = '1') then
      Z <= "10";
    elsif (S(1) = '1') then
      Z <= "01";
    else Z <= "00";
    end if;
  end process;
end if_arch;
```



Comparison to a conditional assignment statement

- An if statement is somewhat like a concurrent conditional signal assignment statement.

```
sig <= value_exp1 when boolean_exp1 else  
      value_exp2 when boolean_exp2 else  
      value_exp3 when boolean_exp3 else  
      ...  
      value_expn;
```

Process(...)

begin

if boolean_exp1 **then** sig <= value_exp1;

elsif boolean_exp2 **then** sig <= value_exp2;

elsif boolean_exp

...

else sig <= value_

end if;

end process;

sig <= value_exp1 **when** boolean_exp1 **else**

value_exp2 **when** boolean_exp2 **else**

value_exp3 **when** boolean_exp3 **else**

...

value_expn;

- The above two statements are equivalent.
- In general, equivalent only if each branch of the if statement consists of a single assignment of the same single signal.

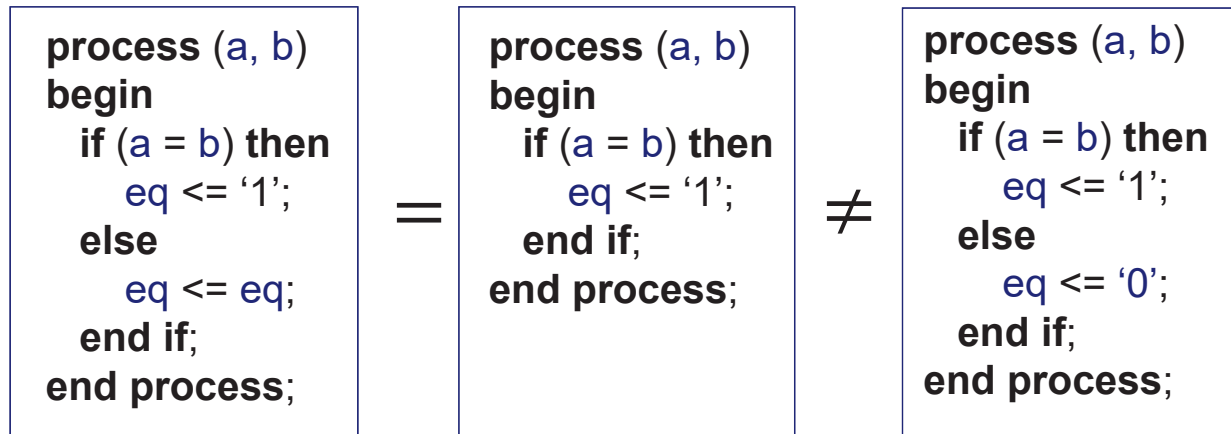
- An if statement allows for arbitrary nesting of if statement.

```
process (CTRL1, CTRL2)
begin
  if CTRL1 = '1' then
    if CTRL2 = '0' then
      MUX_OUT <= "0010";
    else
      MUX_OUT <= "0001";
    end if;
  else
    if CTRL2 = '0' then
      MUX_OUT <= "1000";
    else
      MUX_OUT <= "0100";
    end if;
  end if;
end process;
```

**Write the
equivalent of
conditional signal
assignment**

Incomplete if branch

- In VHDL, only the then branch is mandatory and the other branches can be omitted.



Imply a circuit with a closed feedback loop,
which constitutes memory

Imply a combinational
circuit

Incomplete signal assignment

```
process (a, b)
begin
  if (a > b) then
    gt <= '1';
  elsif (a=b) then
    eq <= '1';
  else
    lt <= '1';
  end if;
end process;
```

ImPLY a circuit with
memories

≠

```
process (a, b)
begin
  if (a > b) then
    gt <= '1';
    eq <= '0';
    lt <= '0';
  elsif (a=b) then
    gt <= '0';
    eq <= '1';
    lt <= '0';
  else
    gt <= '0';
    eq <= '0';
    lt <= '1';
  end if;
end process;
```

=

```
process (a, b)
begin
  gt <= '0';
  eq <= '0';
  lt <= '0';
  if (a > b) then
    gt <= '1';
  elsif (a=b) then
    eq <= '1';
  else
    lt <= '1';
  end if;
end process;
```

ImPLY a combinational
circuit

4.2.4 Case statement

- A case statement is a multiway branch based on the value of a control expression
- General form of a case statement

case *expression* **is**

when *choices* =>

{*sequential-statements*}

{ **when** *choices* =>

{ *sequential-statements* } }

end case;

integer type, enumerated type,
one dimensional character array type
such as BIT_VECTOR.

Must have the same type as the expression.
either a static expression or a static range.

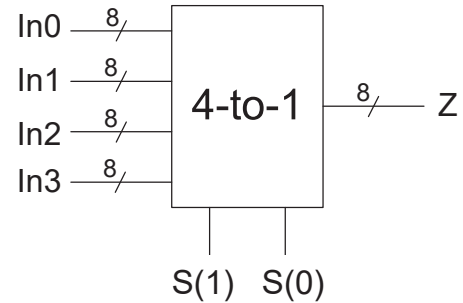
All possible value in the range of *expression*'s type must be covered by one and only one choice.

The final choice can be others, which match all remaining choices in the range of *expression*'s type.

Choices之间需要互斥且包含所有的情况

4-to-1 multiplexer based on a case statement

```
architecture case_arch of mux4 is
begin
  process(In0, In1, In2, In3, S)
  begin
    case S is
      when "00" =>
        Z <= In0;
      when "01" =>
        Z <= In1;
      when "10" =>
        Z <= In2;
      when others =>
        Z <= In3;
    end case;
  end process;
end case_arch;
```



4-to-2 priority encoder based on a case statement

```
architecture case_arch of pr_encoder is
```

```
begin
```

```
  process(S)
```

```
  begin
```

```
    case S is
```

```
      when "1000" | "1001" | "1010" | "1011" |  
          "1100" | "1101" | "1110" | "1111" =>
```

```
        Z <= "11";
```

```
      when "0100" | "0101" | "0110" | "0111" =>
```

```
        Z <= "10";
```

```
      when "0010" | "0011" =>
```

```
        Z <= "01";
```

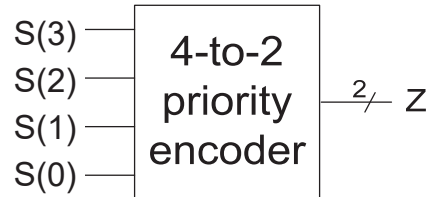
```
      when others =>
```

```
        Z <= "00";
```

```
    end case;
```

```
  end process;
```

```
end case_arch;
```



```

signal S1 : integer range 0 to 7;
Signal OU : bit;
signal I1, I2, I3 : bit;
.....
select_process: process(S1, I1, I2, I3) is
begin
    case S1 is
        when 0 | 2 => OU <= '0' ;
        when 1 => OU <= I1;
        when 3 to 5 => OU <= I2;
        when others => OU <= I3;
    end case ;
end process select_process;

```

Example:
A two-
process
half-adder
model

```
library IEEE;
use IEEE.std_logic_1164.all;
entity half_adder is
port (x, y : in std_logic; sum, carry : out std_logic);
end entity half_adder;
architecture behavior of half_adder is
begin
```

```
    sum_proc: process (x, y) is -- this process computes the value of sum
begin
```

```
    if (x=y) then sum <= '0' after 5 ns;
    else sum <= (x or y) after 5 ns;
    end if;
```

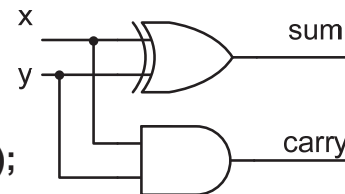
```
end process sum_proc;
```

```
    carry_proc: process (x,y) is -- this process computes the value of carry
begin
```

```
    case x is
    when '0' => carry <= x after 5 ns;
    when '1' => carry <= y after 5 ns;
    when others => carry <= 'X' after 5 ns;
    end case;
```

```
end process carry_proc;
```

```
end architecture behavior;
```



给process起名字

4.2.5 NULL statement

- The statement

 null; 需要写在process里面

is a sequential statement that does not cause any action to take place;

```
variable SEL      : integer range 0 to 31;
variable V        : integer range 0 to 31;
case SEL is
  when 0 to 15 => V := SEL;
  when others => null;
end case;
```